

Unit Testing, TDD & DI

C# Developer Reference — Sessions 3 & 4

xUnit · Moq · TDD · IoC Container

Quick Reference

Topic	Key concept	Remember
Unit Testing	Test one method at a time	AAA: Arrange, Act, Assert
[Fact] / [Theory]	Single test / Multiple data sets	[InlineData] for multiple cases
Naming	MethodName_Condition_ExpectedResult	Read test name = know what it tests
TDD	Write test first, code after	Red → Green → Refactor
Moq	Mock dependencies for isolation	Mock, Setup, Verify, Times
DI	Inject via constructor, not new()	Depend on interface, not concrete
IoC Container	Framework creates dependencies	AddScoped / AddTransient / AddSingleton

Session 3 — Unit Testing & TDD

Project Setup

```
# Create solution

dotnet new sln -n MyProject

# Create main project (classlib — no Main method needed)

dotnet new classlib -n MyProject.Core

# Create test project

dotnet new xunit -n MyProject.Tests

# Link test project to main project

dotnet add MyProject.Tests/MyProject.Tests.csproj reference MyProject.Core/MyProject.Core.csproj

# Add both to solution

dotnet sln add MyProject.Core/MyProject.Core.csproj

dotnet sln add MyProject.Tests/MyProject.Tests.csproj

# Run tests

dotnet test
```

Required packages in test .csproj

```
<ItemGroup>

  <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.9.0" />

</ItemGroup>
```

```

    <PackageReference Include="xunit" Version="2.9.3" />

    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2">

        <PrivateAssets>all</PrivateAssets>

        <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
    </IncludeAssets>

    </PackageReference>

</ItemGroup>

```

■ *Main project must be `OutputType=Library` (not `Exe`). Test project must **NOT** have a `Main` method — remove it if Visual Studio adds one. Without `Microsoft.NET.Test.Sdk`, `dotnet test` will build but run zero tests.*

AAA Pattern — the structure of every test

Every unit test follows three steps: Arrange (prepare data), Act (call the method), Assert (verify the result).

```

public class Calculator
{
    public int Add(int a, int b) => a + b;

    public int Divide(int a, int b)
    {
        if (b == 0) throw new DivideByZeroException("Cannot divide by zero");

        return a / b;
    }
}

public class CalculatorTests

```

```
{

    [Fact]

    public void Add_WhenGivenTwoNumbers_ReturnsCorrectSum()

    {

        // Arrange – prepare

        var calculator = new Calculator();

        int a = 2, b = 3;

        // Act – call the method

        int result = calculator.Add(a, b);

        // Assert – verify

        Assert.Equal(5, result);

    }

    [Fact]

    public void Divide_WhenDivisorIsZero_ThrowsException()

    {

        var calculator = new Calculator();

        // Act + Assert for exceptions

        Assert.Throws<DivideByZeroException>(() => calculator.Divide(10, 0));

    }

}
```

```

    // [Theory] – run same test with multiple data sets

    [Theory]

    [InlineData(2, 3, 5)]

    [InlineData(-1, -1, -2)]

    [InlineData(0, 0, 0)]

    public void Add_WithMultipleCases_ReturnsCorrectSum(int a, int b, int expected)

    {

        var calculator = new Calculator();

        int result = calculator.Add(a, b);

        Assert.Equal(expected, result);

    }

}

```

Test Naming Convention

Every test name must answer: what method, what condition, what result.

```

// Pattern: MethodName_Condition_ExpectedResult

Add_WhenGivenTwoNumbers_ReturnsCorrectSum

Divide_WhenDivisorIsZero_ThrowsException

CreateUser_WhenEmailIsEmpty_ThrowsArgumentException

PlaceOrder_WhenQuantityIsZero_ReturnsBadRequest

```

Common Assert Methods

```

Assert.Equal(expected, actual);           // values are equal

```

```
Assert.NotEqual(expected, actual);           // values are not equal

Assert.True(condition);                      // condition is true

Assert.False(condition);                     // condition is false

Assert.Null(object);                         // object is null

Assert.NotNull(object);                      // object is not null

Assert.Contains(substring, string);          // string contains substring

Assert.IsType<T>(object);                    // object is of type T

Assert.Throws<T>(() => method());             // method throws exception T
```

■ *One test, one assertion — or at least one concern. If a test has 6 assertions and one fails, the rest never run. You won't know which other assertions would have failed.*

TDD — Test-Driven Development

"Write the test first. Write the code after."

TDD cycle: Red → Green → Refactor. Repeat for every small feature.

■ **Red** — Write a test for the feature. It fails because the code doesn't exist yet. This is correct — expected behaviour.

■ **Green** — Write the minimum code to make the test pass. Don't optimise. Don't clean up. Just pass the test.

■ **Refactor** — Clean up the code. Improve naming, remove duplication, apply SOLID. Run tests again — they must still pass.

Example — StringCalculator built with TDD

[illegible]


```
    if (string.IsNullOrEmpty(numbers)) return 0;

    return numbers.Split(',').Select(int.Parse).Sum();
}
```

■ *TDD is not about testing — it's about design. Writing tests first forces you to think about what the method should do before writing how it does it. Senior developers use TDD to avoid over-engineering.*

Moq — Mocking Dependencies

When testing a class that depends on other services (email, database), you don't want to actually send emails or connect to a real database. Moq creates fake objects that pretend to be real services.

Install Moq

```
dotnet add package Moq
```

Core Moq concepts

```
// 1. Create a mock – a fake object implementing the interface

var mockEmail = new Mock<IEmailService>();

// 2. .Object – get the actual fake object to inject

var service = new OrderService(mockEmail.Object);

// 3. Setup – define what the mock returns when called

mockEmail.Setup(e => e.Send("test@gmail.com")).Returns(true);

// 4. Verify – check that the method was called

mockEmail.Verify(e => e.Send("test@gmail.com"), Times.Once);

// 5. Times – how many times the method should have been called

Times.Once      // exactly 1 time

Times.Never     // never called

Times.Exactly(3) // exactly 3 times

// 6. It.IsAny<T>() – match any value of type T

mockRepo.Verify(r => r.Save(It.IsAny<string>(), It.IsAny<int>()), Times.Never);
```

Full example — testing OrderService

```
public class OrderServiceTests
{
    [Fact]

    public void PlaceOrder_WhenValid_SavesAndSendsEmail()
    {
        // Arrange

        var mockRepo = new Mock<IOrderRepository>();

        var mockEmail = new Mock<IEmailService>();

        var service = new OrderService(mockRepo.Object, mockEmail.Object);

        // Act

        service.PlaceOrder("Laptop", 1, "trung@gmail.com");

        // Assert – verify repo was called with correct args

        mockRepo.Verify(
            r => r.Save("Laptop", 1, "trung@gmail.com"),
            Times.Once
        );

        // Assert – verify email was sent

        mockEmail.Verify(
            e => e.Send("trung@gmail.com", It.IsAny<string>()),
```

```

        Times.Once

    );

}

[Fact]

public void PlaceOrder_WhenQuantityIsZero_ThrowsAndNeverSaves()

{

    var mockRepo = new Mock<IOrderRepository>();

    var mockEmail = new Mock<IEmailService>();

    var service = new OrderService(mockRepo.Object, mockEmail.Object);

    // Act + Assert

    Assert.Throws<ArgumentException>(

        () => service.PlaceOrder("Laptop", 0, "trung@gmail.com")

    );

    // Repo must never be called when validation fails

    mockRepo.Verify(

        r => r.Save(It.IsAny<string>(), It.IsAny<int>(), It.IsAny<string>()),

        Times.Never

    );

}

}

```

■ *Moq only works because of Dependency Injection. If OrderService creates its own EmailService with new(), there is no way to inject a mock. DI is the prerequisite for testable code.*

Session 4 — Dependency Injection & IoC

Three concepts to distinguish

Concept	What it is	Example
DIP	SOLID principle — depend on abstraction not concrete	Use IEmailService not SmtplibEmailService
IoC	Principle — don't create your own dependencies	Let something else create them for you
DI	Implementation of IoC — inject via constructor	public OrderService(IEmailService email)

■ Without DI

```
// ■ No DI — tightly coupled, cannot mock, cannot swap

public class OrderService

{

    private readonly EmailService _email = new EmailService(); // hardcoded

}
```

■ With DI

```
// ■ With DI — inject via constructor

public class OrderService

{

    private readonly IEmailService _email;

    public OrderService(IEmailService email) // injected from outside

    {

        _email = email;

    }

}
```

IoC Container in ASP.NET Core

In large apps, manually creating and wiring dependencies is tedious. The IoC Container does it automatically.

```
// Program.cs – register dependencies once

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<IEmailService, SmtplibEmailService>();

builder.Services.AddScoped<IOrderRepository, OrderRepository>();

builder.Services.AddScoped<IOrderValidator, OrderValidator>();

builder.Services.AddScoped<OrderService>();

var app = builder.Build();

app.MapControllers();

app.Run();

// Controller – framework injects automatically, no new() needed

public class OrderController : ControllerBase
{
    private readonly OrderService _orderService;

    public OrderController(OrderService orderService) // auto-injected
    {
        _orderService = orderService;
    }
}
```

```

[HttpPost]

public IActionResult PlaceOrder([FromBody] OrderRequest order)
{
    try
    {
        _orderService.PlaceOrder(order);

        return Ok(order);
    }

    catch (ArgumentException ex)
    {
        return BadRequest(ex.Message);
    }
}
}

```

Service Lifetimes

When registering a service, choose how long each instance lives.

Lifetime	Created when	Use for
Transient	Every injection	Lightweight stateless services
Scoped	Every HTTP request	DB context, repositories
Singleton	Once, app lifetime	Config, cache, logger

```

builder.Services.AddTransient<IEmailService, SmtplibEmailService>(); // new every time

builder.Services.AddScoped<IOrderRepository, OrderRepository>(); // new per request

```



```
builder.Services.AddSingleton<IAppConfig, AppConfig>();           // one forever
```

■ *SOLID + DI + Testing are one system, not three separate topics. SOLID tells you how to design classes. DI lets you swap implementations without changing code. Testing is only possible because DI lets you inject mocks. Remove any one of them and the other two lose most of their value.*